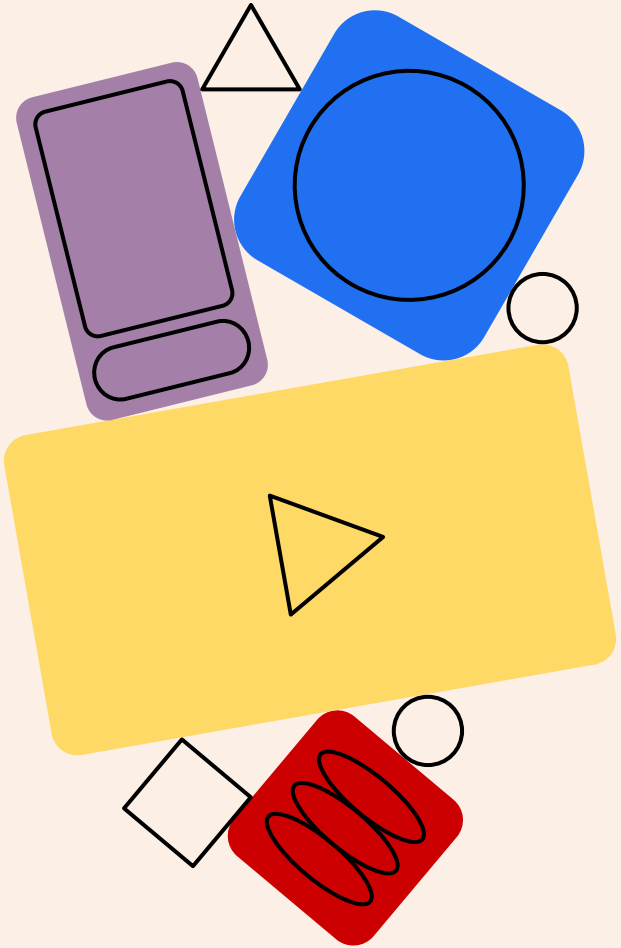
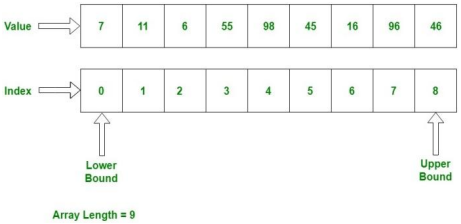
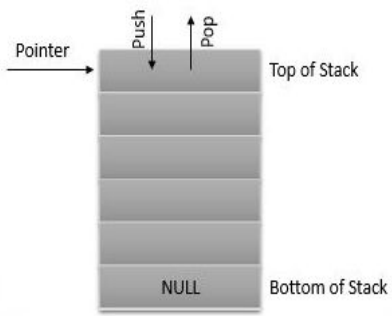
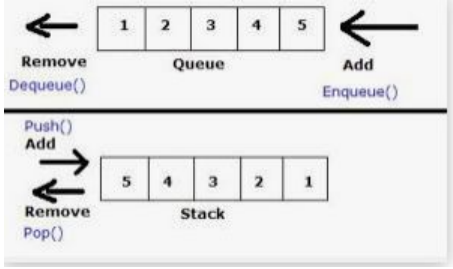
Data Structures

Linear Data Structures - STACK

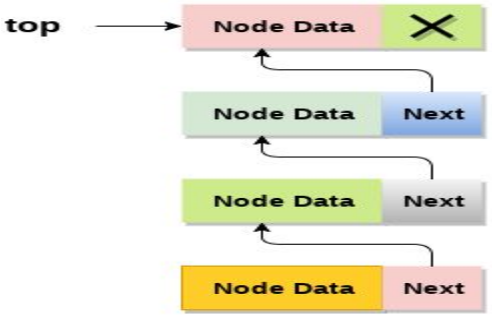


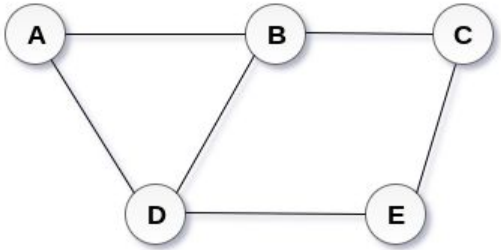

Value
7
11
6
55
98
45
16
96
46
Index
0
1
2
3
4
5
6
7
8
Lower Bound
Upper Bound
Array Length = 9

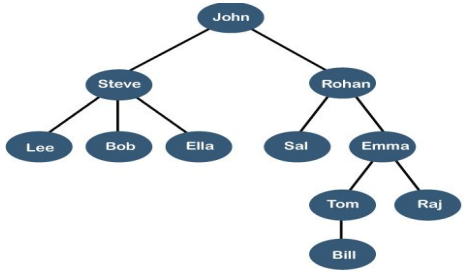

Pointer
Push
Pop
Top of Stack
NULL
Bottom of Stack


1
2
3
4
5
Remove
Dequeue()
Queue
Add
Enqueue()
Push()
Add
5
4
3
2
1
Remove
Pop()
Stack

STACK


top
Node Data
Node Data
Next
Node Data
Next
Node Data
Next


A
B
C
D
E


John
Steve
Rohan
Lee
Bob
Ella
Sal
Emma
Tom
Raj
Bill

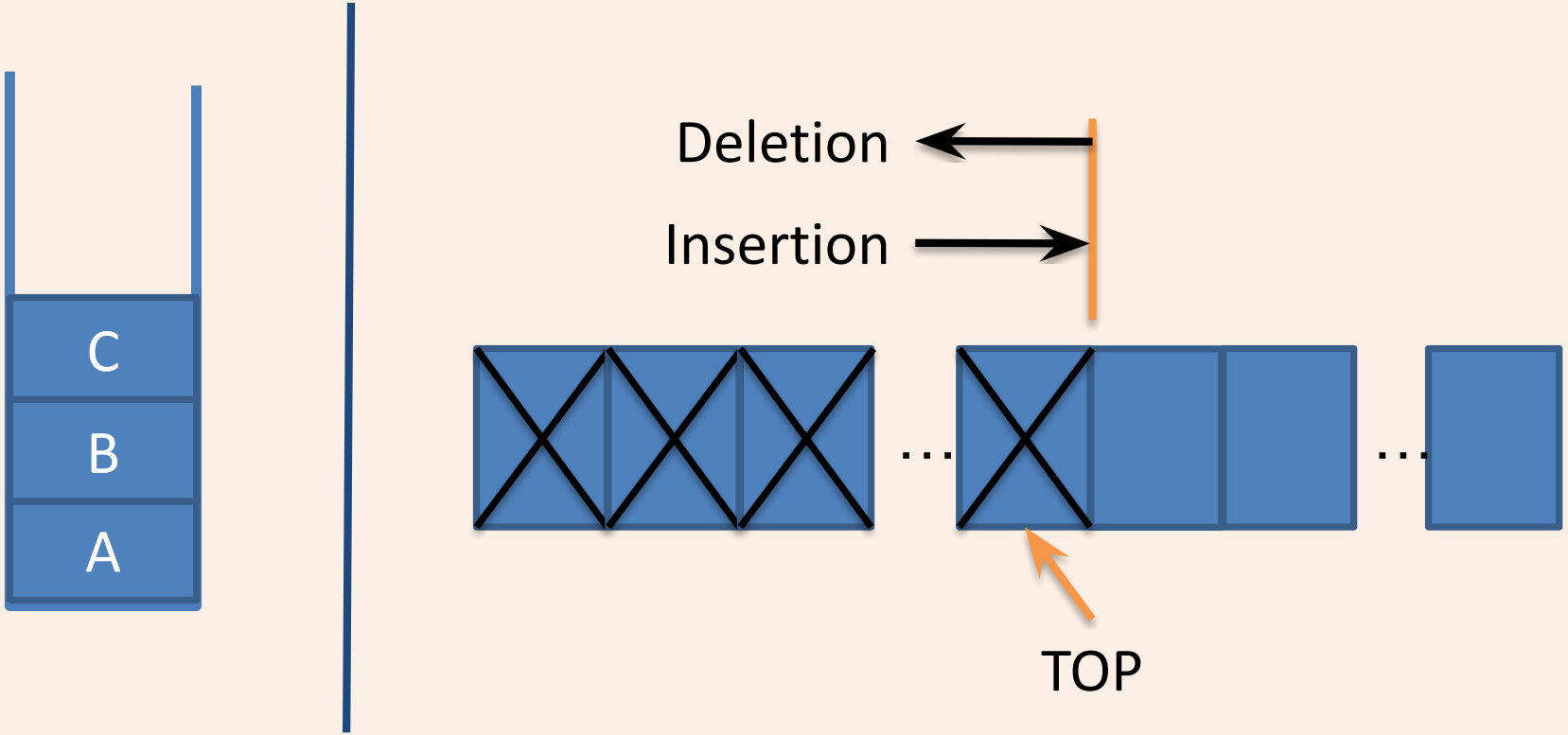
Introduction

- A stack is a collection of objects that are inserted and removed according to the last-in, first-out (LIFO) principle.
- A user may insert objects into a stack at any time, but may only access or remove the most recently inserted object that remains (at the so-called “top” of the stack).
- The name “stack” is derived from the metaphor of a stack of plates in a spring-loaded, cafeteria plate dispenser.
- The fundamental operations involve the “pushing” and “popping”.
- When we need a new plate from the dispenser, we “pop” the top plate off the stack, and when we add a plate, we “push” it down on the stack to become the new top plate.

Introduction

- A linear list which allows insertion and deletion of an element at one end only is called **stack**.
- The insertion operation is called as **PUSH** and deletion operation as **POP**.
- The most accessible elements in stack is known as **top**.
- The elements can only be removed in the opposite orders from that in which they were added to the stack.
- Such a linear list is referred to as a **LIFO (Last In First Out)** list.

Introduction



Introduction

- A pointer **TOP** keeps track of the top element in the stack.
- Initially, when the **stack is empty**, **TOP** has a value of **“zero”**.
- Each time a new element is inserted in the stack, the pointer is **incremented by “one”** before, the element is placed on the stack.
- The pointer is **decremented by “one”** each time a deletion is made from the stack.
- Stacks are the simplest of all data structures, yet they are also among the most important.
- They are used in a host of different applications, and as a tool for many more sophisticated data structures and algorithms.

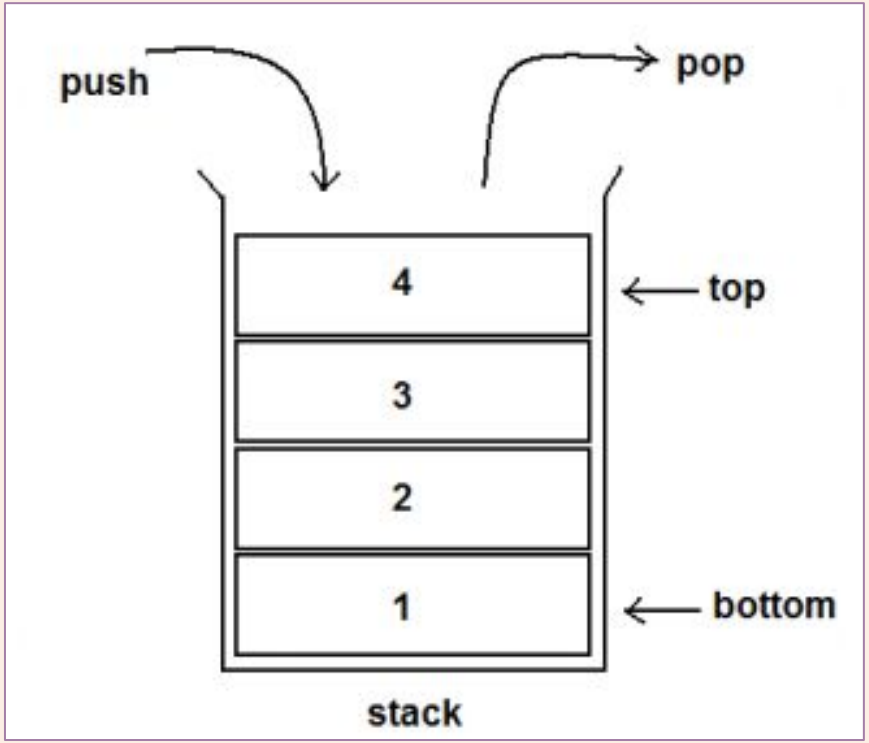
Applications

- Recursion
- Keeping track of function calls
- Evaluation of expressions
- Reversing characters
- Servicing hardware interrupts
- Solving combinatorial problems using backtracking
- Expression Conversion (Infix to Postfix, Infix to Prefix)
- Microsoft Word (Undo / Redo)
- Browser Next/ Previous Pages
- Compiler – Parsing syntax & expression
- etc…

Operations - (General)

- PUSH
- POP
- PEEK/ PEAK
- IS_EMPTY
- IS_FULL

- PEEP - *
- DISPLAY
- CHANGE - *
- SEARCH



Procedure : PUSH (S, TOP, X)

- This procedure inserts an element **X** to the top of a stack.
- Stack is represented by a vector **S** containing **N** elements.
- A pointer **TOP** represents the top element in the stack.

```
1. [Check for stack overflow]
      If     TOP ≥ N
      Then       write ('STACK OVERFLOW')
             Return
2. [Increment TOP]
      TOP ← TOP + 1
3. [Insert Element]
      S[TOP] ← X
4. [Finished]
      Return
```

Stack is empty, TOP = 0, N=3

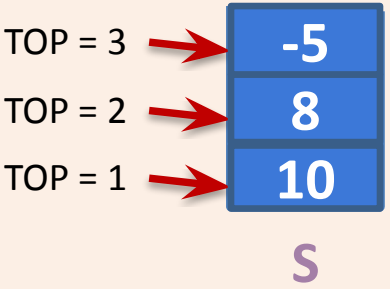
PUSH(S, TOP, 10)

PUSH(S, TOP, 8)

PUSH(S, TOP, -5)

PUSH(S, TOP, 6)

Overflow



Procedure : POP (S, TOP)

- This function ***removes & returns*** the top element from a stack.
- Stack is represented by a vector **S** containing **N** elements.
- A pointer **TOP** represents the top element in the stack.

```
1.  [Check for stack underflow]
    If    TOP = 0
    Then      write ('STACK UNDERFLOW')
              Return (0)
2. [Decrement TOP]
    TOP ← TOP - 1
3. [Return former top element of stack]
    Return(S[TOP + 1])
```

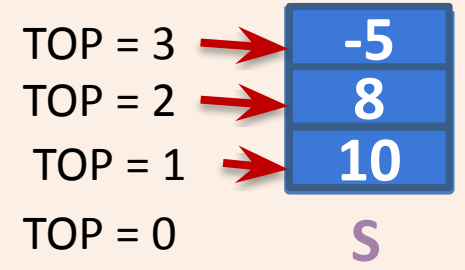
POP(S, TOP)

POP(S, TOP)

POP(S, TOP)

POP(S, TOP)

Underflow



Procedure : PEEP (S, TOP, I)

- This function returns the value of the I^{th} element from the **TOP** of the stack. The element is not deleted by this function.
- Stack is represented by a vector **S** containing **N** elements.

1. [Check for stack underflow]

```
If    TOP-I+1 ≤ 0
Then  write ('STACK UNDERFLOW')
      Return (0)
```

2. [Return I^{th} element from top of the stack]

```
Return(S[TOP–I+1])
```

PEEP (S, TOP, 2)

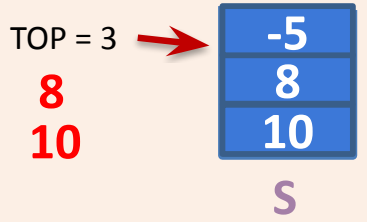
8

PEEP (S, TOP, 3)

10

PEEP (S, TOP, 4)

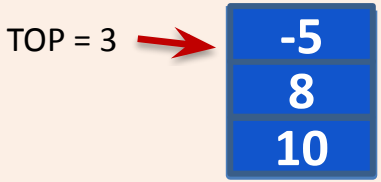
Underflow



Procedure : CHANGE (S, TOP,X,I)

- This procedure changes the value of the **Ith** element from the top of the stack to **X**.
- Stack is represented by a vector **S** containing **N** elements.

```
1.  [Check for stack underflow]
    If    TOP-I+1 ≤ 0
    Then      write ('STACK UNDERFLOW')
              Return
2. [Change I^th element from top
   of the stack]
    S[TOP–I+1] ← X
3. [Finished]
    Return
```



CHANGE (S, TOP, 50, 2)

CHANGE (S, TOP, 9, 3)

CHANGE (S, TOP, 25, 8)

Underflow

Stack Abstract Data Type

- Stack is an abstract data type (ADT) such that an instance S supports the following two methods:
  - S.push(e): Add element e to the top of stack S.
  - S.pop( ): Remove and return the top element from the stack S; an error occurs if the stack is empty.
- Additionally, let us define the following accessor methods for convenience:
  - S.top( ): Return a reference to the top element of stack S, without removing it; an error occurs if the stack is empty.
  - S.is empty( ): Return True if stack S does not contain any elements.
  - len(S): Return the number of elements in stack S; in Python, we implement this with the special method len .
- By convention, we assume that a newly created stack is empty, and that there is no a priori bound on the capacity of the stack. Elements added to the stack can have arbitrary type.

Illustration

- The table shows a series of stack operations and their effects on an initially empty stack S of integers.

Operation	Return Value	Stack Contents
S.push(5)	–	[5]
S.push(3)	–	[5, 3]
len(S)	2	[5, 3]
S.pop( )	3	[5]
S.is_empty( )	False	[5]
S.pop( )	5	[ ]
S.is_empty( )	True	[ ]
S.pop( )	"error"	[ ]
S.push(7)	–	[7]
S.push(9)	–	[7, 9]
S.top( )	9	[7, 9]
S.push(4)	–	[7, 9, 4]
len(S)	3	[7, 9, 4]
S.pop( )	4	[7, 9]
S.push(6)	–	[7, 9, 6]
S.push(8)	–	[7, 9, 6, 8]
S.pop( )	8	[7, 9, 6]

Implementation in Python

- Array
  - List Class
  - Array Class from Array Module
  - Advanced
    - collections.deque (Not Discussed)
    - queue.LifoQueue (Not Discussed)
- Linked List

Implementation in Python

- We can implement a stack quite easily by storing its elements in a Python list.
- The list class already supports adding an element to the end with the append method, and removing the last element with the pop method, so it is natural to align the top of the stack at the end of the list.
- Although a programmer could directly use the list class in place of a formal stack class, lists also include behaviors (e.g., adding or removing elements from arbitrary positions) that would break the abstraction that the stack ADT represents.
- Also, the terminology used by the list class does not precisely align with traditional nomenclature for a stack ADT, in particular the distinction between append and push.
- Instead, we can use a list for internal storage while providing a public interface consistent with a stack - **Adapter Pattern**.

Implementation in Python

- The adapter design pattern applies to any context where we effectively want to modify an existing class so that its methods match those of a related, but different, class or interface.
- One general way to apply the adapter pattern is to define a new class in such a way that it contains an instance of the existing class as a hidden field, and then to implement each method of the new class using methods of this hidden instance variable.
- By applying the adapter pattern in this way, we have created a new class that performs some of the same functions as an existing class, but repackaged in a more convenient way.

Implementation in Python

- In the context of the stack ADT, we can adapt Python's list class using the correspondences shown in Table.

Stack Method	*Realization with Python list*
S.push(e)	L.append(e)
S.pop( )	L.pop( )
S.top( )	L[−1]
S.is_empty( )	len(L) == 0
len(S)	len(L)

Expression Conversion & Evaluation

- Infix, Postfix and Prefix notations are three different but equivalent notations of writing algebraic expressions.

Infix Notation

- Infix, Postfix and Prefix notations are three different but equivalent notations of writing algebraic expressions.
- While writing an arithmetic expression using infix notation, the operator is placed between the operands.
- For example, A+B; here, plus operator is placed between the two operands A and B.
- Although it is easy to write expressions using infix notation, computers find it difficult to parse as they need a lot of information to evaluate the expression.
- Information is needed about operator precedence, associativity rules, and brackets which overrides these rules.
- So, computers work more efficiently with expressions written using prefix and postfix notations.

Postfix Notation

- Prefix notation known as Polish notation and a postfix notation which is better known as Reverse Polish Notation or RPN.
- In postfix notation, the operator is placed after the operands. For example, if an expression is written as A+B in infix notation, the same expression can be written as AB+ in postfix notation.
- The order of evaluation of a postfix expression is always from left to right.
- The expression (A + B) * C is written as:
  AB+C* in the postfix notation.
- A postfix operation does not even follow the rules of operator precedence. The operator which occurs first in the expression is operated first on the operands.
- For example, given a postfix notation AB+C*. While evaluation, addition will be performed prior to multiplication.

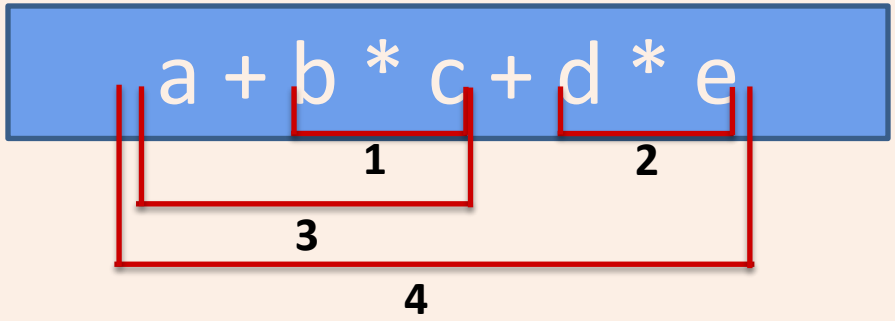
Prefix Notation

- In a prefix notation, the operator is placed before the operands.
- For example, if A+B is an expression in infix notation, then the corresponding expression in prefix notation is given by +AB.
- While evaluating a prefix expression, the operators are applied to the operands that are present immediately on the right of the operator.
- Prefix expressions also do not follow the rules of operator precedence, associativity, and even brackets cannot alter the order of evaluation.
- The expression (A + B) * C is written as:

  *+ABC in the prefix notation

Polish Expression & their Compilation

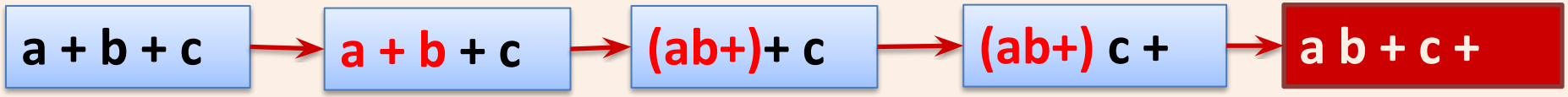
- Evaluating Infix Expression



- A **repeated scanning** **from left to right is needed** as operators appears inside the operands.
- ***Repeated scanning is avoided*** if the **infix expression** is first **converted** to an equivalent parenthesis free ***prefix or suffix (postfix) expression***.
- **Prefix Expression: Operator**, Operand, Operand
- **Postfix Expression:** Operand, Operand, **Operator**

Polish Notation

Sr.	Infix	Postfix	Prefix
1	a	a	a
2	a + b	a b +	+ a b
3	a + b + c	a b + c +	+ + a b c
4	a + (b + c)	a b c + +	+ a + b c
5	a + (b * c)	a b c * +	+ a * b c
6	a * (b + c)	a b c + *	* a + b c
7	a * b * c	a b * c *	* * a b c



Evaluation of postfix expression

- Each **operator** in **postfix** string **refers** to the *previous two operands* in the string.
- Each time we **read** an **operand**, we **PUSH** it onto **Stack**.
- When we reach an **operator**, its **operands** will be **top two elements** on the stack.
- We can then **POP** these two elements, perform the indicated operation on them and PUSH the result on the stack so that it will available for use as an operand of the next operator.

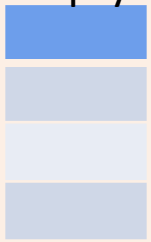
Algorithm: EVALUAE_POSTFIX

1. Add) to postfix expression.
2. Read postfix expression Left to Right until) encountered
3. If **operand** is encountered, push it onto Stack
   [End If]
4. If **operator** is encountered, Pop two elements
   i) A -> Top element
   ii) B -> Next to Top element
   iii) **Evaluate** B operator A
   push B operator A onto Stack
5. Set result = pop
6. END

Evaluation of postfix expression

Evaluate Expression: 5 6 2 - +

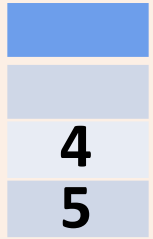
Empty Stack



Read 5, it is operand? PUSH
Read 6, it is operand? PUSH
Read 2, it is operand? PUSH

Read - , it is operator? POP two symbols and perform operation and PUSH result

Operand 1 - Operand 2



Read + , it is operator? POP two symbols and perform operation and PUSH result

Operand 1 + Operand 2

Read next symbol, if is it is end of string, POP answer from Stack

Answer

Algorithm: EVALUAE_POSTFIX

```
1. [Initialize Stack]
    TOP □ 0
    VALUE □ 0

2. [Evaluate the postfix expression]
    Repeat until last character
        TEMP □ NEXTCHAR (POSTFIX)
        If   TEMP is DIGIT
        Then PUSH (S, TOP, TEMP)
        Else OPERAND2 □ POP (S, TOP)
             OPERAND1 □ POP (S, TOP)
             VALUE □ PERFORM_OPERATION(OPERAND1,
                                       OPERAND2, TEMP)
             PUSH (S, TOP, VALUE)

3. [Return answer from stack]
   Return (POP (S, TOP))
```

Algorithm: Converting Infix to Postfix

Let, X is an arithmetic expression written in infix notation. This algorithm finds the equivalent postfix expression Y.

1. Push “(“onto Stack, and add “)” to the end of X.
2. Scan X from **left to right** and repeat Step 3 to 6 for each element of X until the Stack is empty.
3. If an **operand** is encountered, add it to Y.
4. If a **left parenthesis** is encountered, push it onto Stack.
5. If an **operator** is encountered ,then:
   i. Repeatedly pop from Stack and add to Y each operator (on the top of Stack) which has the same precedence as or higher precedence than operator.
   ii. Add operator to Stack.
   [End of If]

Algorithm: Converting Infix to Postfix

6. If a right parenthesis is encountered ,then:

   i. Repeatedly pop from Stack and add to Y each operator (on the top of Stack) until a left parenthesis is encountered.

   ii. Remove the left Parenthesis.
   [End of If]
   [End of If]

7. END.

Algorithm Infix to Prefix

1. Push “)” onto STACK, and add “(“ to end of the A
2. Scan A from right to left and repeat step 3 to 6 for each element of A until the STACK is empty
3. If an operand is encountered add it to B
4. If a right parenthesis is encountered push it onto STACK
5. If an operator is encountered then:
    i. Repeatedly pop from STACK and add to B each operator (on the top of STACK) which has same or higher precedence than the operator.
    ii. Add operator to STACK
6. If left parenthesis is encountered then
    I. Repeatedly pop from the STACK and add to B (each operator on top of stack until a left parenthesis is encountered)
    II. Remove the left parenthesis
7. Exit

Arithmetic Priority

Parentheses() -- 1
Exponent(^,$,|) – 2
Multiplication/ division – 3
Addition/Substraction -- 4

In case of equal precedence/priority of operators ,expression solved from left to right.

Infix	Postfix	Prefix
A+B*C	ABC*+	+A*BC
((A-(B+C))*D)|(E+F)	ABC+-D*EF+|	|*-A+BCD+EF
(A+B)*(C-D)$E*F	AB+CD-E$*F*	**+AB$-CDEF

Example(((A-(B+C))*D)|(E+F)

- Post fix

  ((A-BC+)*D)|(E+F)

  (ABC+- * D)|(E+F)

  ABC+-D* | (E+F)

  ABC+-D* | EF+

  ABC+-D*EF+|

- Prefix

  ((A - +BC)*D)|(E+F)

  (-A+BC * D) |(EF)

  *-A+BCD |(E+F)

  *-A+BCD | +EF

  |*-A+BCD+EF

POSTFIX Evaluation

- In ***Postfix*** notation the expression is scanned from left to right.
- When a number is seen it is pushed onto the stack;

- when an operator is seen the operator is applied to the two numbers popped from the stack and the result is pushed back to the stack.

Stack example: postfix notation

- Postfix notation
  - Operations are done by specifying operands, then the operator
  - Example: 2 3 4 + * results in 14
    - Calculates 2 * (3 + 4)
- Postfix evalution implemented with a stack
  - When we see a operand (number), push it on the stack
  - When we see an operator
    - Pop the appropriate number of operands off the stack
    - Do the calculation
    - Push the result back onto the stack
  - At the end, the stack should have the (one) result of the calculation

Postfix Evaluation

Operand: push
Operator: pop 2 operands, do the math, pop result back onto stack

1 2 3 + *

	Postfix	Stack(bot -> top)
a)	1 2 3 + *	
b)	2 3 + *	1
c)	3 + *	1 2
d)	+ *	1 2 3
e)	*	1 5 // 5 from 2 + 3
f)		5 // 5 from 1 * 5

Evaluation of postfix expressions

Steps

given the postfix expression, get a token and:

1) If the token is an operand, push its value on the (value) stack.
2) If the token is an operator, pop two values from the stack and apply that operator to them ,perform calculation; then push the result back on the stack.

Example 1: A B * C D E / - +

Let A = 5, B = 3, C = 6, D = 8, E = 2.

	value stack
push (5)	5
push (3)	5 3
push (pop * pop)	15
push (6)	15 6
push (8)	15 6 8
push (2)	15 6 8 2
push (pop / pop)	15 6 4

Infix to Postfix conversion

Infix Expression: 3 + 2 * 4

Postfix Expression:

Operator Stack:

Simple Example

Infix Expression: + 2 * 4

PostFix Expression: 3

Operator Stack:

Simple Example

Infix Expression: 2 * 4

PostFix Expression: 3

Operator Stack: +

Simple Example

Infix Expression: * 4

PostFix Expression: 3 2

Operator Stack: +

Simple Example

Infix Expression: 4

PostFix Expression: 3 2

Operator Stack: + *

Simple Example

Infix Expression:

PostFix Expression: 3 2 4

Operator Stack: + *

Simple Example

Infix Expression:

PostFix Expression: 3 2 4 *

Operator Stack: +

Simple Example

Infix Expression:

PostFix Expression: 3 2 4 * +

Operator Stack:

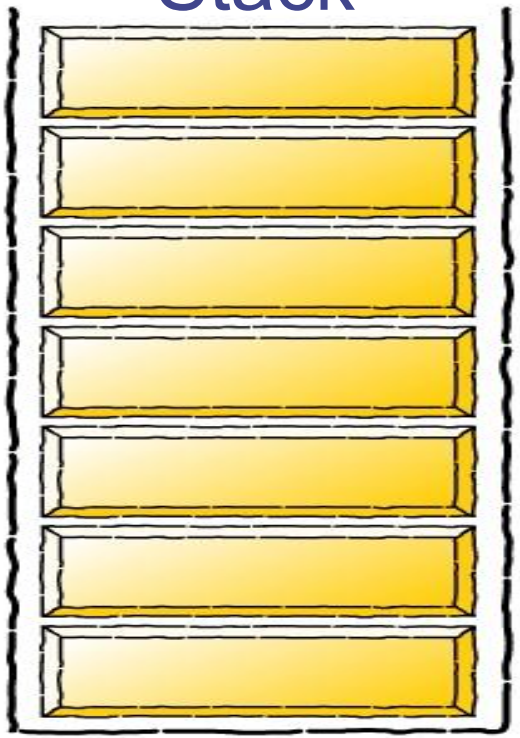
Infix to postfix Conversion

- Rule

1. Push "(" onto Stack, and add ")" to the end of infix expression.
2. Scan infix expression from left to right and repeat Step 3 to 6 for each element of X until the Stack is empty.
3. If an operand is encountered, add it to postfix expression.
4. If a left parenthesis is encountered, push it onto Stack.
5. If an operator is encountered ,then:
    1. Repeatedly pop from Stack and add to postfix expression each operator (on the top of Stack) which has the same precedence as or higher precedence than operator.
    2. Add operator to Stack.
    [End of If]
6. If a right parenthesis is encountered ,then:
    1. Repeatedly pop from Stack and add to postfix expression each operator (on the top of Stack) until a left parenthesis is encountered.
    2. Remove the left Parenthesis.
    [End of If]
7. END

Infix to postfix conversion

Stack

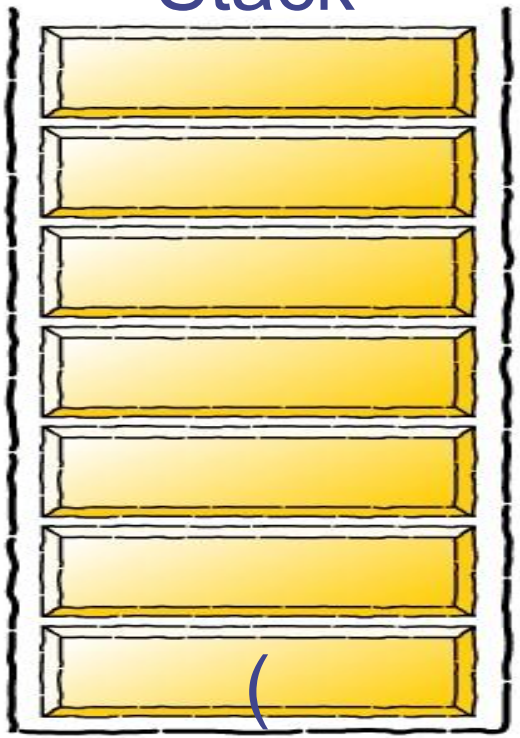


Infix Expression

(a + b - c) * d – (e + f)

Postfix Expression

Stack

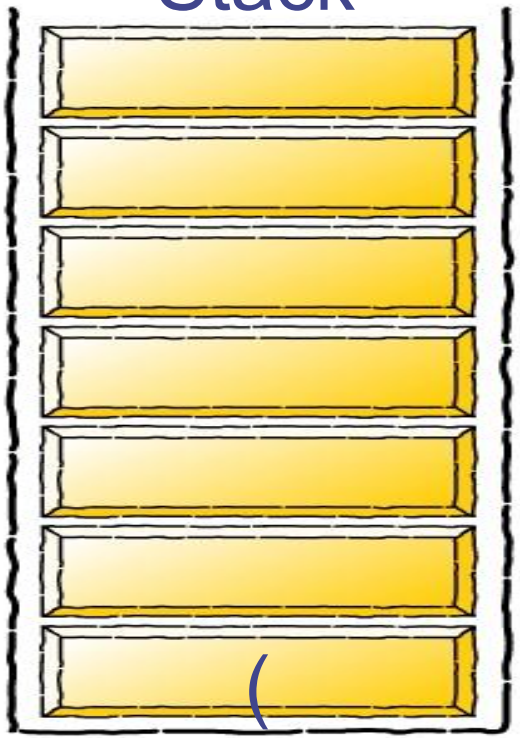


Infix Expression

a + b - c) * d – (e + f)

Postfix Expression

Stack

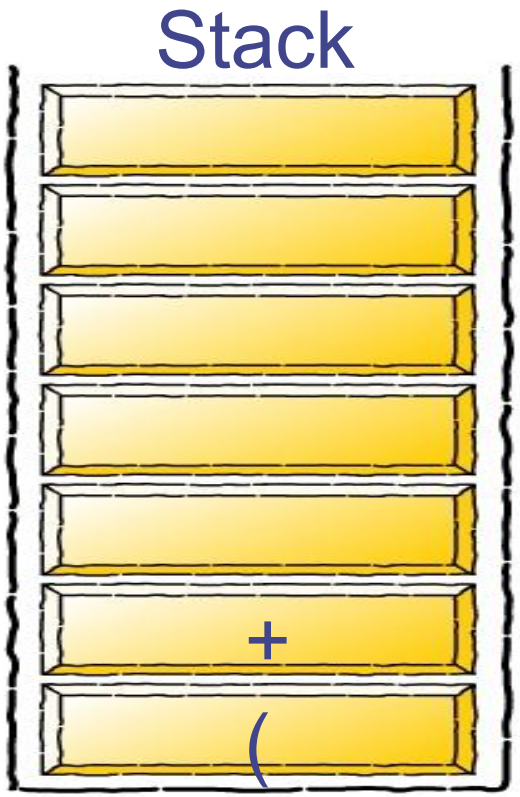


Infix Expression

+ b - c) * d – (e + f)

Postfix Expression

a

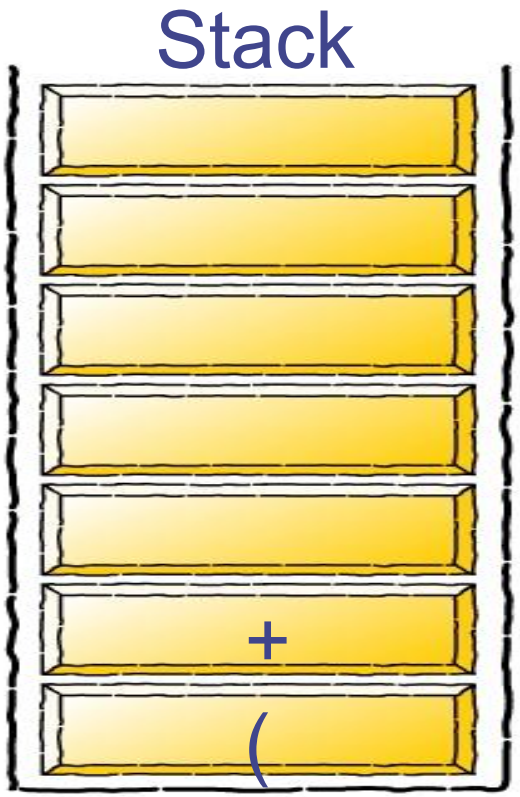


Infix Expression

b - c) * d – (e + f)

Postfix Expression

a



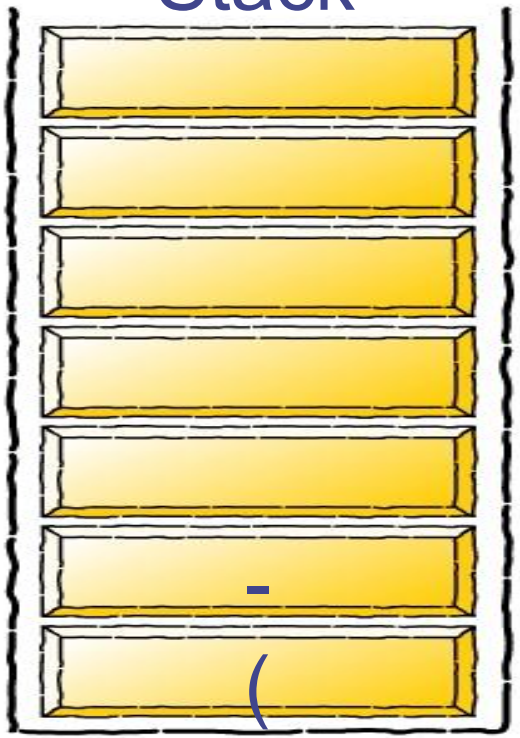
Infix Expression

- c) * d – (e + f)

Postfix Expression

a b

Stack

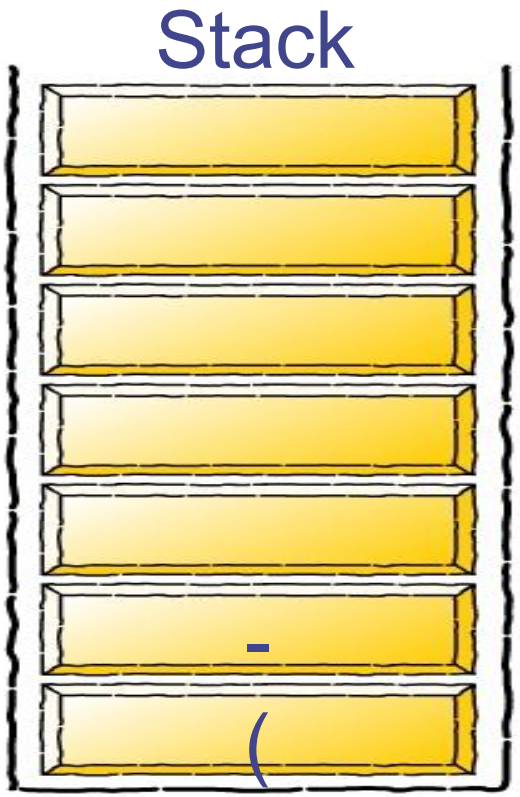


Infix Expression

c) * d – (e + f)

Postfix Expression

a b +



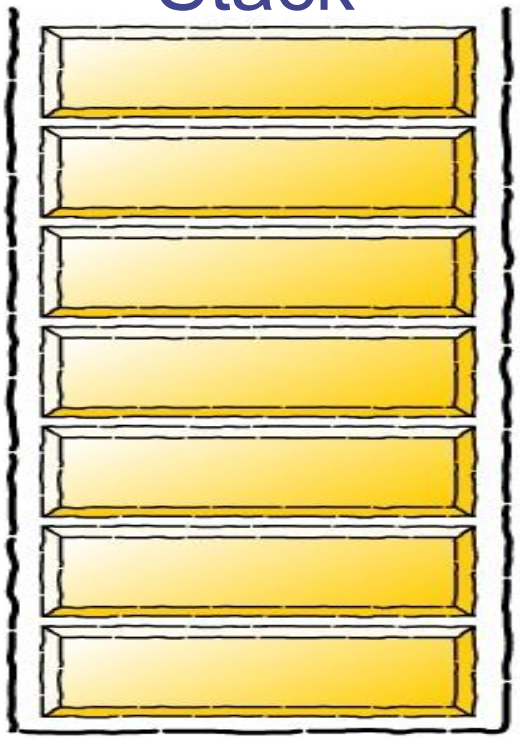
Infix Expression

) * d – (e + f)

Postfix Expression

a b + c

Stack



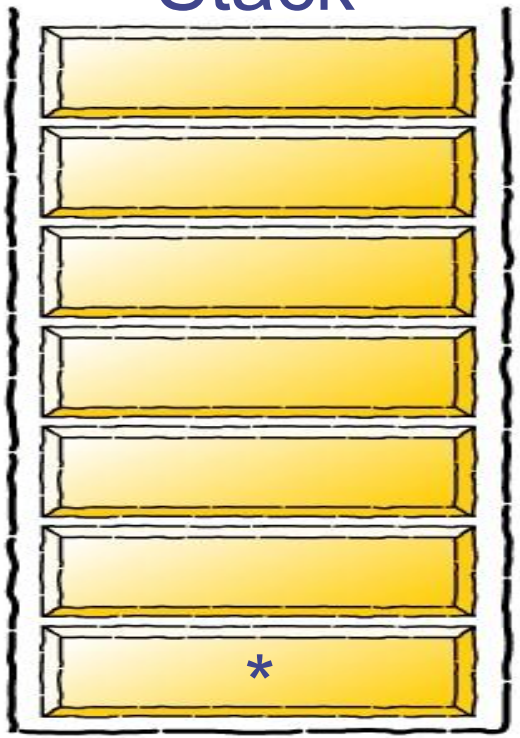
Infix Expression

* d – (e + f)

Postfix Expression

a b + c -

Stack



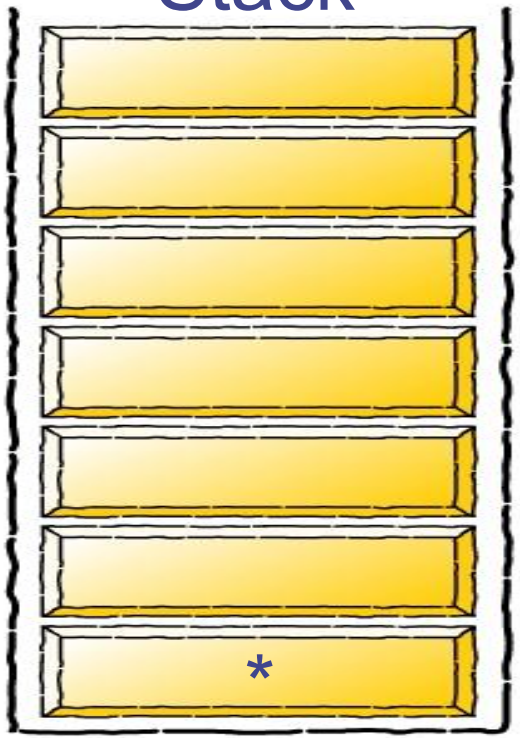
Infix Expression

d – (e + f)

Postfix Expression

a b + c -

Stack

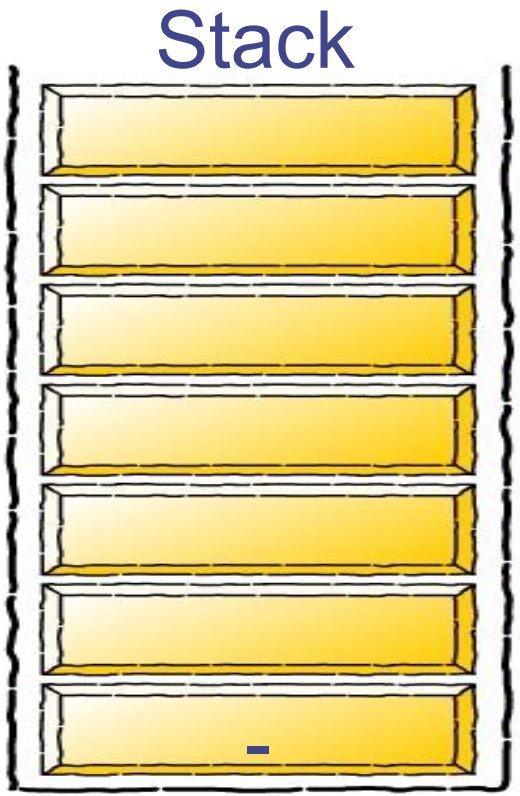


Infix Expression

– (e + f)

Postfix Expression

a b + c - d

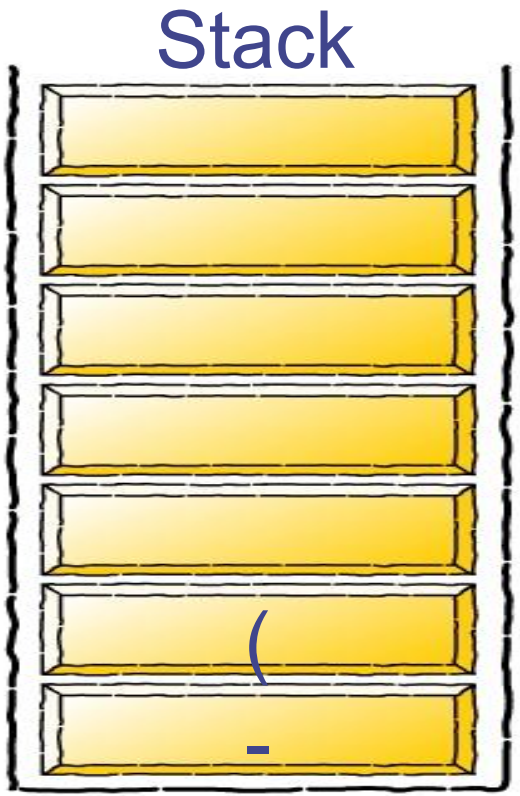


Infix Expression

(e + f)

Postfix Expression

a b + c – d *



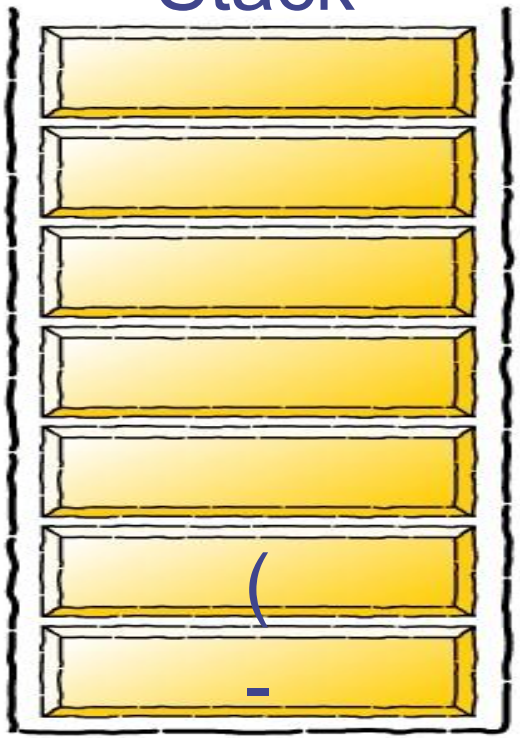
Infix Expression

e + f)

Postfix Expression

a b + c – d *

Stack

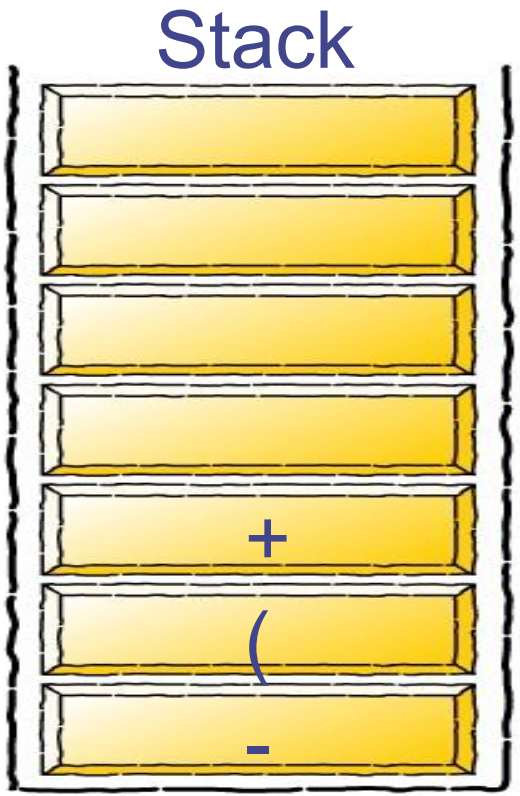


Infix Expression

+ f)

Postfix Expression

a b + c – d * e

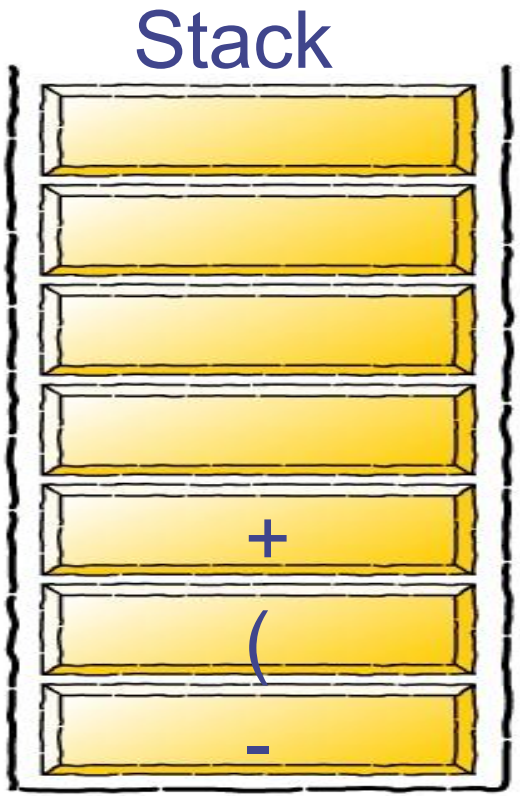


Infix Expression

f)

Postfix Expression

a b + c – d * e



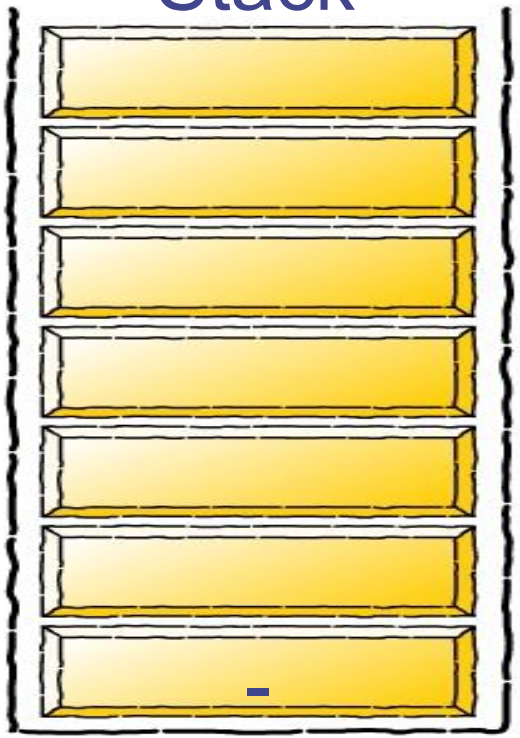
Infix Expression

)

Postfix Expression

a b + c – d * e f

Stack

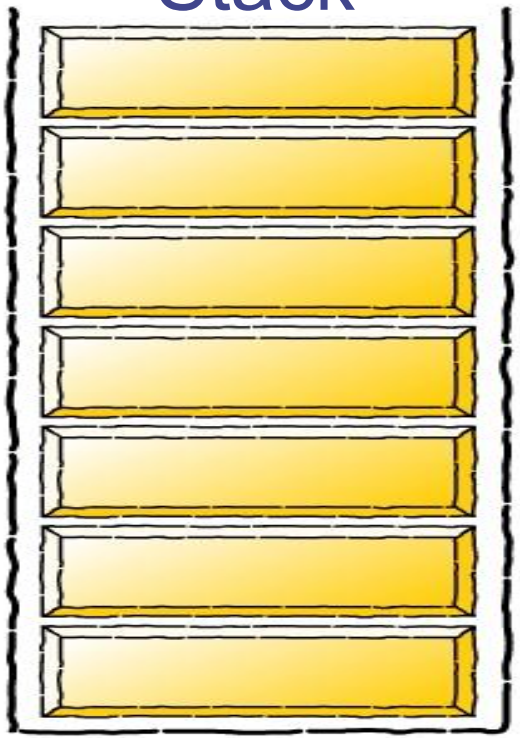


Infix Expression

Postfix Expression

a b + c – d * e f +

Stack



Infix Expression

Postfix Expression

a b + c – d * e f + -

Example : Translate the following infix form to a postfix form: A * B + (C - D / E)

Infix queue	Operator stack	Postfix queue
A		A
*	*	A
B	*	A B
+	+	A B *
(	(+	A B *
C	(+	A B * C
-	- (+	A B * C
D	- (+	A B * C D
/	/ - (+	A B * C D
E	/ - (+	A B * C D E
)	+	A B * C D E / -
		A B * C D E / - +

Converting Expression from Infix to Prefix (Algorithm)

1) Reverse the input string.
2) Examine the next element in the input.
3) If it is operand, add it to output string.
4) If it is Closing parenthesis, push it on stack.
5) If it is an operator, then
   i) If stack is empty, push operator on stack.
   ii) If the top of stack is closing parenthesis, push operator on stack.
   iii) If it has same or higher priority than the top of stack, push operator on stack.
   iv) Else pop the operator from the stack and add it to output string, repeat step 5.
6) If it is a opening parenthesis, pop operators from stack and add them to output string until a closing parenthesis is encountered. Pop and discard the closing parenthesis.
7) If there is more input go to step 2
8) If there is no more input, unstack the remaining operators and add them to output string.
9) Reverse the output string.

Example

- Suppose we want to convert
  2*3/(2-1)+5*(4-1) into Prefix form:
  Reversed Expression:)1-4(*5+)1-2(/3*2

Char Scanned	Stack Contents (Top on right)	Prefix Expression (right to left)
)	)	
1	)	1
-	)-	1
4	)-	14
(	Empty	14-
*	*	14-
5	*	14-5
+	+	14-5*
)	+)	14-5*

1	+)	14-5*1
-	+)-	14-5*1
2	+)-	14-5*12
(	+	14-5*12-
/	+/	14-5*12-
3	+/	14-5*12-3
*	+/*	14-5*12-3
2	+/*	14-5*12-32
	Empty	14-5*12-32*/+

- Reverse the output string : +/*23-21*5-41
- So, the final Prefix Expression is +/*23-21*5-41


Thank You